ISA Design

Abdullah All Mamun Siam
Lecturer, Dhaka International University(DIU)

Instruction Set Architecture (ISA)

The Instruction Set Architecture (ISA)/Architecture is the part of the processor that is visible to the programmer or compiler writer. The ISA specifies the behavior of machine code running on implementations of that ISA. The ISA serves as the boundary between software and hardware.

In general, an ISA defines the supported data types, the registers, the hardware support for managing main memory, fundamental features (such as the memory consistency, addressing modes, virtual memory), and the input/output model of a family of implementations of the ISA.22

For example, ISA of Intel/AMD CPU is x86-64 architecture, ISA of ARM CPU is ARM architecture etc.

Instruction Set Architecture (ISA)

Format of ISA is usually like:

ISA design depends on several criteria:

1. Word size of CPU
   (Ex: 2-bit/4-bit/8-bit etc.)
2. No. of registers
   (Ex: 2 registers/4 registers/ 16 registers etc.
3. No. of ALU operations supported
   (Ex: ALU supports 8/10/16 operations etc.)
4. Types of instructions supported
   (Ex: Arithmetic, Logic, Branching, Memory operations etc.)

ISA Design (Opcode)

Opcode of our ISA will depend on two things:

1. Types of instructions supported
2. No. of ALU operations supported

Our CPU supports 4 types of instructions:

1. Arithmetic & Logic Instructions (Register Mode)
2. Arithmetic & Logic Instructions (Immediate Mode)
3. Branching
4. Memory & IO Operations

Our ALU supports 12 operations total: AND, OR, XOR, ADD, SUB, SHL, SHR, MUL, ROL, ROR, NOT and DIV.

ISA Design (Opcode)

So, our CPU has:

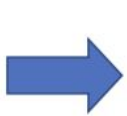
1. Types of instructions supported (4) → $\log_2 4 = 2$
2. No. of ALU operations supported (12) → $\log_2 12 \approx 4$

Format of ISA is usually like:

Opcode	Operands

So, Opcode will be 6 bits in our ISA.

2 bits	4 bits
Types of instruction	Operations (ALU selection lines)



6 bits
Opcode

ISA Design (Operands)

Our operands of ISA design will depend on two things:
1. No. of registers in register set
2. Word size of CPU

No. of registers in CPU is 8. So, no of bits need to address 8 registers is $\log_2 8 = 3$ and Word size of CPU is 4-bits. So, size of value will be 4-bits too.

For Arithmetic & Logic instructions (Register Mode), consider
`ADD R2, R3`
So, we need to select two registers (3 bits each).

For Arithmetic & Logic instructions (Immediate Mode), consider
`ADD R2, 3`
So, we need to select one register (3 bits) and one value (4 bits).

ISA Design (Operands)

So, our CPU has:

1. No. of registers in register set (**8**) → $\log_2 8 = 3$
2. Word size of CPU (**4**) → 4 bits (value)

Format of ISA is usually like:

Opcode	Operands

For Arithmetic & Logic instructions (Register Mode),

3 bits	3 bits
Register 1	Register 2

→

6 bits
Operands

For Arithmetic & Logic instructions (Immediate Mode),

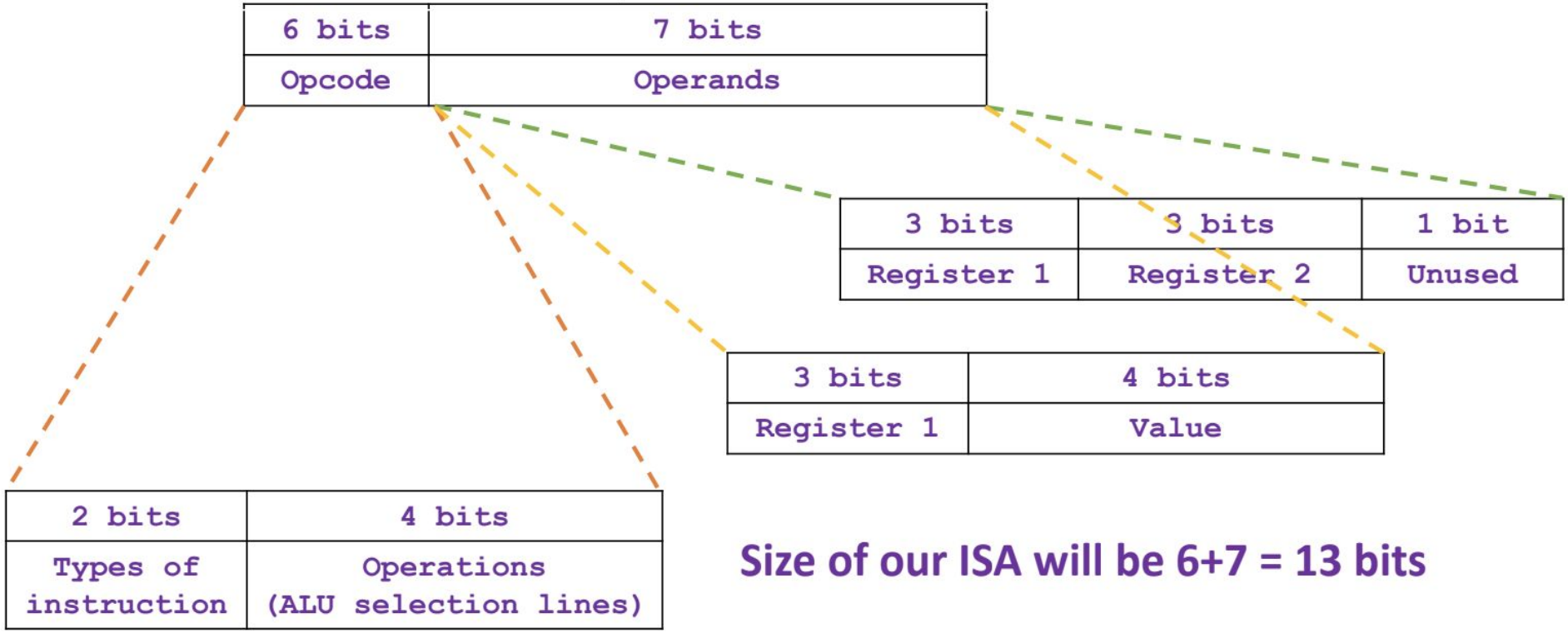
3 bits	4 bits
Register 1	Value

→

7 bits
Operands

ISA Design (Operands)

Format of ISA is usually like:



Size of our ISA will be 6+7 = 13 bits

Types of Instruction

There will be 4 types of instruction:

Types of instruction	Opcode (First 2 bits)	Example Assembly

Arithmetic & Logic Instruction (Register Mode)	00	`ADD R0, R1`
		`XOR R2, R3`
		`SHL R5, 1`

Arithmetic & Logic Instruction (Immediate Mode)	01	`ADD R0, 5` `XOR R2, 6`
Branching	10	`JMP LABEL` `JC LABEL`

Memory and Input/Output Operations	11	`LOAD R0, [R1]`
		`STORE R1, 10`
		`ACCEPT_INPUT`

ISA of Arithmetic & Logic Instruction (Register Mode)

Opcode (6 bit)		Register 1	Register 2	Unused
2 bits	4 bits	3 bits	3 bits	1 bit
Types of instruction	Operations (ALU selection lines)	Ra (000-111)	Rb (000-111)	X

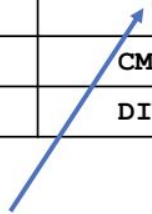
Convention,
Ra = Ra OP Rb

12 7	6 4	3 1	0
00XXXX (Opcode)	Register 1	Register 2	Unused

ISA of Arithmetic & Logic Instruction (Register Mode)

Opcode		Register 1	Register 2	Assembly Example
Type (2 bits)	Operations (4 bits)	3 bits	3 bits	
00	0000 (AND)	000-100 (R0-R4)	000-100 (R0-R4)	AND R0, R1
	0001 (OR)	000-100 (R0-R4)	000-100 (R0-R4)	OR R1, R2
	0010 (XOR)	000-100 (R0-R4)	000-100 (R0-R4)	XOR R2, R3
	0011 (SHL)	000-100 (R0-R4)	000-100 (R0-R4)	SHL R3, R1
	0100 (SHR)	000-100 (R0-R4)	000-100 (R0-R4)	SHR R4, R2
	0101 (ADD)	000-100 (R0-R4)	000-100 (R0-R4)	ADD R4, R4
	0110 (SUB)	000-100 (R0-R4)	000-100 (R0-R4)	SUB R4, R4
	0111 (MUL)	000-100 (R0-R4)	000-100 (R0-R4)	MUL R3, R4
	1000 (ROL)	000-100 (R0-R4)	000-100 (R0-R4)	ROL R0, R2
	1001 (ROR)	000-100 (R0-R4)	000-100 (R0-R4)	ROR R1, R2
	1010 (NOT)	000-100 (R0-R4)	000-100 (R0-R4)	NOT R4
	1011 (CMP)	000-100 (R0-R4)	000-100 (R0-R4)	CMP R4, R2
	1100 (DIV)	000-100 (R0-R4)	000-100 (R0-R4)	DIV R4, R2

Note: R5-R7 are Special Purpose Registers. That's why they are not included.



This instruction do operation on single register. (They do not need 2nd register [3:1])

Example: ISA of Arithmetic & Logic Instruction (Register Mode)

12	11	10	09	08	07	06	05	04	03	02	01	00
00XXXX (Opcode)						Register 1			Register 2			Unused

Question:

Translate following assembly code to machine code using ISA:

1. `XOR R2, R3`

Answer:

Since type of instruction is Arithmetic & Logic (Register Mode), so opcode of first two bits[12:11] is 00.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	x	x	x	x	x	x	x	x	x	x	x

Since selection lines of XOR operation in ALU is 0010, so rest of opcode[10:7] is 0010

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	1	0	x	x	x	x	x	x	x

Since first register is R2, it will be register 1[6:4] in ISA and its value in binary is 010.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	1	0	0	1	0	x	x	x	x

Since second register is R3, it will be register 2[3:1] in ISA and its value in binary is 011.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	1	0	0	1	0	0	1	1	x

Unused bit[0] will not be used in this operation. It can be anything 0/1.

Example: ISA of Arithmetic & Logic Instruction (Register Mode)

12	11	10	09	08	07	06	05	04	03	02	01	00
00XXXX (Opcode)						Register 1			Register 2			Unused

Question:

Translate following assembly code to machine code using ISA:

2. `CMP R2, R3`

Answer:

Since type of instruction is Arithmetic & Logic (Register Mode), so opcode of first two bits[12:11] is 00.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	x	x	x	x	x	x	x	x	x	x	x

Since selection lines of XOR operation in ALU is 1011, so rest of opcode[10:7] is 1011.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	1	0	1	1	x	x	x	x	x	x	x

Since first register is R2, it will be register 1[6:4] in ISA and its value in binary is 010.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	1	0	1	1	0	1	0	x	x	x	x

Since second register is R3, it will be register 2[3:1] in ISA and its value in binary is 011.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	1	0	1	1	0	1	0	0	1	1	x

ISA of Arithmetic & Logic Instruction (Immediate Mode)

Opcode (6 bit)		Register 1	Constant
2 bits	4 bits	3 bits	4 bits
Types of instruction	Operations (ALU selection lines)	Ra (000-111)	Value (0000-1111)

Convention,

Ra = Ra OP Constant

12 7	6 4	3 0
01XXXX (Opcode)	Register 1	Value

ISA of Arithmetic & Logic Instruction (Immediate Mode)

Opcode		Register 1	Constant	Assembly Example
Type (2 bits)	Operations (4 bits)	3 bits	4 bits	
01	0000 (AND)	000-100 (R0-R4)	0000-1111 (0-15)	AND R0, 1
	0001 (OR)	000-100 (R0-R4)	0000-1111 (0-15)	OR R1, 2
	0010 (XOR)	000-100 (R0-R4)	0000-1111 (0-15)	XOR R2, 3
	0011 (SHL)	000-100 (R0-R4)	0000-1111 (0-15)	SHL R3, 3 (MAX 3)
	0100 (SHR)	000-100 (R0-R4)	0000-1111 (0-15)	SHR R4, 2(MAX 3)
	0101 (ADD)	000-100 (R0-R4)	0000-1111 (0-15)	ADD R4, 4
	0110 (SUB)	000-100 (R0-R4)	0000-1111 (0-15)	SUB R3, 5
	0111 (MUL)	000-100 (R0-R4)	0000-1111 (0-15)	MUL R1, 3 (MAX 3)
	1000 (ROL)	000-100 (R0-R4)	0000-1111 (0-15)	ROL R0, 1 (MAX 3)
	1001 (ROR)	000-100 (R0-R4)	0000-1111 (0-15)	ROR R1, 2 (MAX 3)
	1010 (NOT)	000-100 (R0-R4)	0000-1111 (0-15)	NOT R4
	1011 (CMP)	000-100 (R0-R4)	0000-1111 (0-15)	CMP R1, 7
	1100 (DIV)	000-100 (R0-R4)	0000-1111 (0-15)	DIV R1, 7

Note: R5-R7 are Special Purpose Registers. That's why they are not included.

These instructions do operations on single register and they do the same thing as they do in register mode. (They do not need value [3:0])

Example: ISA of Arithmetic & Logic Instruction (Immediate Mode)

12	11	10	09	08	07	06	05	04	03	02	01	00
01XXXX (Opcode)						Register 1			Value			

Question:

Translate following assembly code to machine code using ISA:

1. `XOR R2, 3`

Answer:

Since type of instruction is Arithmetic & Logic (Immediate Mode), so opcode of first two bits[12:11] is 01.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	x	x	x	x	x	x	x	x	x	x	x

Since selection lines of XOR operation in ALU is 0010, so rest of opcode[10:7] is 0010

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	0	0	1	0	x	x	x	x	x	x	x

Since first register is R2, it will be register 1[6:4] in ISA and its value in binary is 010.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	0	0	1	0	0	1	0	x	x	x	x

Since value is 3, it will be value[3:0] in ISA and its value in binary is 0011.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	0	0	1	0	0	1	0	0	0	1	1

Example: ISA of Arithmetic & Logic Instruction (Immediate Mode)

12	11	10	09	08	07	06	05	04	03	02	01	00
01XXXX (Opcode)						Register 1			Value			

Question:

3. ROL R4, 3

Answer:

Since type of instruction is Arithmetic & Logic (Immediate Mode), so opcode of first two bits[12:11] is 01.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	x	x	x	x	x	x	x	x	x	x	x

Since selection lines of ROL operation in ALU is 1000, so rest of opcode[10:7] is 1000.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	1	0	0	0	x	x	x	x	x	x	x

Since first register is R4, it will be register 1[6:4] in ISA and its value in binary is 100.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	1	0	0	0	1	0	0	x	x	x	x

Since value is 3, it will be value[3:0] in ISA and its value in binary is 0011.

12	11	10	09	08	07	06	05	04	03	02	01	00
0	1	1	0	0	0	1	1	1	0	0	1	1